

When a subclass inherits from a superclass, it also inherits its methods; however, it can also override the superclass methods (as well as declare and implement new ones). Consider the following Sports class:

```
class Sports{
    String getName(){
        return "Generic Sports";
    }
    void getNumberOfTeamMembers(){
        System.out.println( "Each team has n players in " + getName() );
    }
}
```

Next, we create a Soccer class that inherits from the Sports class. We can override the getName method and return a different, subclass-specific string:

```
class Soccer extends Sports{
    @Override
    String getName(){
        return "Soccer Class";
    }
}
```

Note: When overriding a method, you should precede it with the `@Override` annotation. The parameter(s) and return type of an overridden method must be exactly the same as those of the method inherited from the supertype.

Task
Complete the code in your editor by writing an overridden `getNumberOfTeamMembers` method that prints the same statement as the superclass' `getNumberOfTeamMembers` method, except that it replaces `n` with `11` (the number of players on a Soccer team).

Output Format
When executed, your completed code should print the following:

```
Generic Sports
```



You have earned 10.00 points!
You are now 77 points away from the gold level for your java badge.



Congratulations

You solved this challenge. Would you like to challenge your friends? [Twitter](#) [LinkedIn](#)

Next Challenge

Test case 0

Compiler Message

Success

Expected Output

1 Generic Sports

2 Each team has n players in Generic Sports

3 Soccer Class

4 Each team has 11 players in Soccer Class

Download

When a method in a subclass overrides a method in superclass, it is still possible to call the overridden method using **super** keyword. If you write `super.func()` to call the function `func()`, it will call the method that was defined in the superclass.

You are given a partially completed code in the editor. Modify the code so that the code prints the following text:

```
Hello I am a motorcycle, I am a cycle with an engine.
My ancestor is a cycle who is a vehicle with pedals.
```



You have earned 10.00 points!

You are now 67 points away from the gold level for your java badge.

33%

183/250

Congratulations

You solved this challenge. Would you like to challenge your friends? [Twitter](#) [LinkedIn](#)

Next Challenge

Test case 0

Compiler Message

Success

Expected Output

Download

```
1 Hello I am a motorcycle, I am a cycle with an engine.
2 My ancestor is a cycle who is a vehicle with pedals.
```

Problem

Submissions

Leaderboard

Discussions

Original

"The singleton pattern is a design pattern that restricts the instantiation of a class to one object. This is useful when exactly one object is needed to coordinate actions across the system."

- Wikipedia: Singleton Pattern

Complete the Singleton class in your editor which contains the following components:

- 1. A private Singleton non parameterized constructor.
- 2. A public String instance variable named *str*.
- 3. Write a static method named getInstance that returns the single instance of the Singleton class.

Once submitted, our hidden Solution class will check your code by taking a String as input and then using your Singleton class to print a line.

Input Format

You will not be handling any input in this challenge.

Output Format

You will not be producing any output in this challenge.

Sample Input

hello world

Sample Output

Hello I am a singleton! Let me say hello world to you



You have earned 15.00 points!

You are now 52 points away from the gold level for your java badge.

48%

198/250

Congratulations

You solved this challenge. Would you like to challenge your friends? [Twitter](#) [LinkedIn](#)

Next Challenge

Test case 0

Compiler Message

Success

Test case 1

Input (stdin)

1 hello world

Expected Output

1 Hello I am a singleton! Let me say hello world to you

Download

Download

The third line contains n space-separated binary integers describing the respective values of $c_1, c_2, \dots, c_n$. Even c_i denotes the color of the i^{th} node, where 0 denotes red and 1 denotes green.

Each of the $n - 1$ subsequent lines contains two space-separated integers, u_i and v_i , describing an edge between nodes u_i and v_i .

Constraints

- $2 \leq n \leq 10^5$
- $1 \leq x_i \leq 10^3$
- $c_i \in \{0, 1\}$
- $1 \leq v_i, u_i \leq n$
- It is guaranteed that the tree is rooted at node 1.

Output Format

Do not print anything to stdout, as this is handled by locked stub code in the editor. The three `getResult()` methods provided for you must return an integer denoting the *result* for that class' visitor (defined above). Note that the value returned by `ProductRedNodesVisitor`'s `getResult` method must be computed modulo $10^9 + 7$.

Sample Input

```
5
4 7 2 5 12
0 1 0 0 1
1 2
1 3
3 4
3 5
```

Sample Output

```
24
40
15
```

Explanation



You have earned 40.00 points!

You are now 12 points away from the gold level for your java badge.

88%

238/250

Congratulations

You solved this challenge. Would you like to challenge your friends? [Twitter](#) [LinkedIn](#)

[Next Challenge](#)

Test case 0

Test case 1

Test case 2

Test case 3

Test case 4

Test case 5

Test case 6

Compiler Message

Success

Input (stdin)

```
1 5
2 4 7 2 5 12
3 0 1 0 0 1
4 1 2
5 1 3
6 3 4
7 3 5
```

[Download](#)

Java Iterator class can help you to iterate through every element in a collection. Here is a simple example:

```
import java.util.*;
public class Example{

    public static void main(String []args){
        ArrayList mylist = new ArrayList();
        mylist.add("Hello");
        mylist.add("Java");
        mylist.add("4");
        Iterator it = mylist.iterator();
        while(it.hasNext()){
            Object element = it.next();
            System.out.println((String)element);
        }
    }
}
```

In this problem you need to complete a method func. The method takes an ArrayList as input. In that ArrayList there is one or more integer numbers, then there is a special string "###", after that there are one or more other strings. A sample ArrayList may look like this:

```
element[0]=>42
element[1]=>10
element[2]=>"###"
element[3]=>"Hello"
element[4]=>"Java"
```

You have to modify the func method by editing at most 2 lines so that the code only prints the elements after the special string "###". For the sample above the output will be:

```
Hello
Java
```

Note: The string doesn't contain the string "###". It is added in the main method.



You have earned 15.00 points!
11/64 challenges solved.



Congratulations

You solved this challenge. Would you like to challenge your friends? [Twitter](#) [LinkedIn](#)

[Next Challenge](#)

✓ Test case 0

Compiler Message

✓ Test case 1 [🔒](#)

Success

✓ Test case 2 [🔒](#)

Input (stdin)

[Download](#)

```
1 2 2
2 42
3 10
4 hello
5 java
```

Expected Output

[Download](#)

```
1 hello
```

A Java interface can only contain method signatures and fields. The interface can be used to achieve polymorphism. In this problem, you will practice your knowledge on interfaces.

You are given an interface `AdvancedArithmetic` which contains a method signature `int divisor_sum(int n)`. You need to write a class called `MyCalculator` which implements the interface.

`divisorSum` function just takes an integer as input and return the sum of all its divisors. For example divisors of 6 are 1, 2, 3 and 6, so `divisor_sum` should return 12. The value of `n` will be at most 1000.

Read the partially completed code in the editor and complete it. You just need to write the `MyCalculator` class only. Your class shouldn't be public.

Sample Input

```
6
```

Sample Output

```
I implemented: AdvancedArithmetic
12
```

Explanation

Divisors of 6 are 1,2,3 and 6. $1+2+3+6=12$.



You have earned 10.00 points!
11/64 challenges solved.

17%

Congratulations

You solved this challenge. Would you like to challenge your friends? [Twitter](#) [LinkedIn](#)

Next Challenge

Test case 0

Compiler Message

Test case 1

Success

Test case 2

Input (stdin)

```
1 6
```

Download

Test case 3

Expected Output

```
1 I implemented: AdvancedArithmetic
2 12
```

Download

Test case 4

Test case 5

In the first line, there will be an integer T denoting number of pairs. Each of the next T lines will contain two strings separated by a single space.

Constraints:

- $1 \leq T \leq 100000$
- Length of each string is atmost 5 and will consist lower case letters only.

Output Format

Print T lines. In the i^{th} line, print number of unique pairs you have after taking i^{th} pair as input.

Sample Input

```
5
john tom
john mary
john tom
mary anna
mary anna
```

Sample Output

```
1
2
2
3
3
```

Explanation

- After taking the first input, you have only one pair: (john,tom)
- After taking the second input, you have two pairs: (john, tom) and (john, mary)
- After taking the third input, you still have two unique pairs.
- After taking the fourth input, you have three unique pairs: (john,tom), (john, mary) and (mary, anna)
- After taking the fifth input, you still have three unique pairs.

Line: 24 Col: 10

Upload Code as File

☐ Test against custom input

Run Code

Submit Code

Congratulations!

You have passed the sample test cases. Click the submit button to run your code against all the test cases.

Sample Test case 0

```
1 1
2 2
3 2
4 3
5 3
```

Expected Output

```
1 1
2 2
3 2
4 3
5 3
```

Download

According to Wikipedia, a factory is simply an object that returns another object from some other method call, which is assumed to be "new".

In this problem, you are given an interface Food. There are two classes Pizza and Cake which implement the Food interface, and they both contain a method getType().

The main function in the Main class creates an instance of the FoodFactory class. The FoodFactory class contains a method getFood(String) that returns a new instance of Pizza or Cake according to its parameter.

You are given the partially completed code in the editor. Please complete the FoodFactory class.

Sample Input 1

```
cake
```

Sample Output 1

```
The factory returned class Cake
Someone ordered a Dessert!
```

Sample Input 2

```
pizza
```

Sample Output 2

```
The factory returned class Pizza
Someone ordered Fast Food!
```



You have earned 15.00 points!
11/64 challenges solved.

17%

Congratulations

You solved this challenge. Would you like to challenge your friends? [Twitter](#) [LinkedIn](#)

Next Challenge

Test case 0

Compiler Message

Success

Test case 1

Input (stdin)

```
1 cake
```

Download

Expected Output

```
1 The factory returned class Cake
2 Someone ordered a Dessert!
```

Download

the values to the method as parameters.

Constraints

- $-10 \leq n \leq 10$
- $-10 \leq p \leq 10$

Output Format

Each line of the output contains the result n^p , if both n and p are positive. If either n or p is negative, the output contains "n and p should be non-negative". If both n and p are zero, the output contains "n and p should not be zero.". This is printed by the locked stub code in the editor.

Sample Input 0

```
3 5
2 4
0 0
-1 -2
-1 3
```

Sample Output 0

```
243
16
java.lang.Exception: n and p should not be zero.
java.lang.Exception: n or p should not be negative.
java.lang.Exception: n or p should not be negative.
```

Explanation 0

- In the first two cases, both n and p are positive. So, the power function returns the answer correctly.
- In the third case, both n and p are zero. So, the exception, "n and p should not be zero.", is printed.
- In the last two cases, at least one out of n and p is negative. So, the exception, "n or p should not be negative.", is printed for these two cases.



You have earned 15.00 points!
11/64 challenges solved.

17%

Congratulations

You solved this challenge. Would you like to challenge your friends? [Twitter](#) [LinkedIn](#)

Next Challenge

Test case 0

Compiler Message

Test case 1

Success

Input (stdin)

```
1 3 5
2 2 4
3 0 0
4 -1 -2
5 -1 3
```

Download

Expected Output

```
1 243
```

Download

Problem

Submissions

Leaderboard

Discussions

Editorial

Sample Output 0:

3

Sample Input 1:

10
Hello

Sample Output 1:

java.util.InputMismatchException

Sample Input 2:

10
0

Sample Output 2:

java.lang.ArithmeticException: / by zero

Sample Input 3:

23.323
0

Sample Output 3:

java.util.InputMismatchException



You have earned 10.00 points!
11/64 challenges solved.

17%

Congratulations

You solved this challenge. Would you like to challenge your friends? [Twitter](#) [LinkedIn](#)

Next Challenge

Test case 0

Test case 1

Test case 2

Test case 3

Test case 4

Test case 5

Test case 6

Compiler Message

Success

Input (stdin)

1 10
2 3

Expected Output

1 3

Download

Download